

## Assignment 1

Submitted By :-

Shreyash Tripathi (102303328)

### Q1. DAXPY Loop :-

Code :-

```
#include <chrono>
#include <cstdlib>
#include <limits>
#include <iostream>
#include <omp.h>
#include <random>
#include <vector>
using namespace std;

double get_random_number() {
    static thread_local std::mt19937 mt{std::random_device{}()};
    static thread_local std::uniform_real_distribution<double>
random_range{0.0,1.0};
    return random_range(mt);
}

int main(int argc, char** argv){
    if(argc < 2){
        cout << "Usage: ./daxpy_loop scalar_value\n";
        return EXIT_FAILURE;
    }
    int SIZE{1<<16};
    int totol_num_threads{omp_get_max_threads()};
    double a{atof(argv[1])};
    vector<double> x(SIZE, 0.0);
    vector<double> y(SIZE, 0.0);

    for(int start_threads{2};start_threads <= totol_num_threads;
++start_threads){
        cout << "Using threads: " << start_threads << '\n';
        omp_set_num_threads(start_threads);
        auto start_time{chrono::steady_clock::now()};
        #pragma omp parallel for
```

```
        for(int i = 0;i<SIZE;++i){
            x[i] = get_random_number();
            y[i] = get_random_number();
            x[i] = a*x[i] + y[i];
        }
        auto end_time{chrono::steady_clock::now()};
        auto exec_time{end_time - start_time};
        chrono::duration<double, std::milli> ms{exec_time};
        cout << "Execution time: " << ms.count() << "ms\n";
    }
    return EXIT_SUCCESS;
```

### Program Output :-

```
shreyash@shreyash-Aspire-A715-76G:~/UCS645/LAB1$ ./question1
Usage: ./daxpy_loop scalar_value
shreyash@shreyash-Aspire-A715-76G:~/UCS645/LAB1$ ./question1 35.5
Using threads: 2
Execution time: 1.46697ms
Using threads: 3
Execution time: 1.93018ms
Using threads: 4
Execution time: 1.4274ms
Using threads: 5
Execution time: 1.24483ms
Using threads: 6
Execution time: 1.05587ms
Using threads: 7
Execution time: 1.22847ms
Using threads: 8
Execution time: 1.12101ms
Using threads: 9
Execution time: 0.560014ms
Using threads: 10
Execution time: 0.318067ms
Using threads: 11
Execution time: 12.6612ms
Using threads: 12
Execution time: 11.174ms
```

### Observation :-

After execution of the program we can see that after 8 threads the program performance deteriorates. This happens because my cpu has 8 cores, so when we create 9 or more threads, they cannot all run at the same time. The CPU wastes time constantly shuffling them around to give everyone a turn, and this shuffling takes longer than just letting 8 threads finish the job without interruption.

## Observation from perf stat :-

```
Performance counter stats for './question1 33.5':

    178.45 msec task-clock                #    7.233 CPUs utilized
         22      context-switches         #   123.281 /sec
          6      cpu-migrations            #    33.622 /sec
        455      page-faults              #    2.550 K/sec
  210,711,381    cpu_atom/cycles/          #    1.181 GHz                (23.87%)
  260,573,034    cpu_core/cycles/          #    1.460 GHz                (66.31%)
  173,087,915    cpu_atom/instructions/    #    0.82   insn per cycle    (28.79%)
  195,818,092    cpu_core/instructions/    #    0.75   insn per cycle    (66.31%)
   26,872,321    cpu_atom/branches/        #   150.584 M/sec             (28.97%)
   24,937,002    cpu_core/branches/        #   139.739 M/sec             (66.31%)
   1,552,426     cpu_atom/branch-misses/   #    5.78% of all branches    (28.75%)
   1,539,666     cpu_core/branch-misses/   #    6.17% of all branches    (66.31%)
  TopdownL1 (cpu_core)                   #   54.3 % tma_backend_bound
                                           #   12.9 % tma_bad_speculation
                                           #   11.5 % tma_frontend_bound
                                           #   21.3 % tma_retiring        (66.31%)
  TopdownL1 (cpu_atom)                   #   11.7 % tma_bad_speculation
                                           #   24.3 % tma_retiring        (29.31%)
                                           #   36.2 % tma_backend_bound
                                           #   36.2 % tma_backend_bound aux
                                           #   27.8 % tma_frontend_bound  (29.91%)

 0.024673906 seconds time elapsed

 0.175679000 seconds user
 0.004015000 seconds sys

shreyash@shreyash-Aspire-A715-76G:~/UCS645/LAB1$
```

**Effective Parallelism:** The metric 7.233 CPUs utilized confirms successful multithreading, with the program keeping ~7 cores active simultaneously. This aligns with the optimal execution times observed at 7-8 threads.

**Memory-Bound Bottleneck:** The low IPC (Instructions Per Cycle) of 0.75 indicates the CPU spends significant time waiting for memory access rather than computing.

**Efficient Scheduling:** The 22 count for context-switches and cpu-migrations during this specific run indicates stable thread scheduling without OS interference, which explains why performance was optimal before the thread count exceeded the hardware limits.

## Q2. Matrix Multiplication :-

Code :-

```
#include <chrono>
#include <limits>
#include <iostream>
#include <random>
#include <vector>
using namespace std;

double get_random_number() {
    static thread_local mt19937 mt{random_device{}}();
    static thread_local uniform_real_distribution<double>
random_range{0.0,1.0};
    return random_range(mt);
}

int main() {
    int ROW_SIZE{1000};
    int COL_SIZE{1000};
    vector<double> mat_a(ROW_SIZE*COL_SIZE, 0.0);
    vector<double> mat_b(ROW_SIZE*COL_SIZE, 0.0);
    vector<double> mat_b_T(ROW_SIZE * COL_SIZE, 0.0);
    vector<double> res(ROW_SIZE*COL_SIZE, 0.0);

    #pragma omp parallel for collapse(2)
    for(int i = 0; i<ROW_SIZE; ++i) {
        for(int j = 0; j<COL_SIZE; ++j) {
            mat_a[i*COL_SIZE+j] = get_random_number();
            mat_b[i*COL_SIZE+j] = get_random_number();
        }
    }

    #pragma omp parallel for collapse(2)
    for(int i = 0; i < ROW_SIZE; ++i) {
        for(int j = 0; j < COL_SIZE; ++j) {
            mat_b_T[j * ROW_SIZE + i] = mat_b[i * COL_SIZE +
j];
        }
    }

    cout<< "Using No Threading \n";
```

```

    auto start_time{chrono::steady_clock::now()};
    for(int i = 0;i<ROW_SIZE;++i){
        for(int j = 0;j<COL_SIZE;++j){
            double sum{0.0};
            for(int k = 0;k<ROW_SIZE;++k){
                sum +=
mat_a[i*COL_SIZE+k]*mat_b_T[j*COL_SIZE+k];
            }
            res[i*COL_SIZE+j] = sum;
        }
    }

    auto end_time{chrono::steady_clock::now()};
    auto exec_time{end_time-start_time};
    chrono::duration<double,milli> ms{exec_time};
    cout<<"Execution time:"<<ms.count()<<'\\n';

    cout<<"Using 1D Threading \\n";
    start_time = chrono::steady_clock::now();
    #pragma omp parallel for
    for(int i = 0;i<ROW_SIZE;++i){
        for(int j = 0;j<COL_SIZE;++j){
            double sum{0.0};
            for(int k = 0;k<ROW_SIZE;++k){
                sum +=
mat_a[i*COL_SIZE+k]*mat_b_T[j*COL_SIZE+k];
            }
            res[i*COL_SIZE+j] = sum;
        }
    }

    end_time = chrono::steady_clock::now();
    exec_time = end_time-start_time;
    ms = exec_time;
    cout<<"Execution time:"<<ms.count()<<'\\n';

    cout<<"Using 2D Threading \\n";
    start_time = chrono::steady_clock::now();
    #pragma omp parallel for
    for(int i = 0;i<ROW_SIZE;++i){
        #pragma omp parallel for
        for(int j = 0;j<COL_SIZE;++j){
            double sum{0.0};
            for(int k = 0;k<ROW_SIZE;++k){
                sum +=
mat_a[i*COL_SIZE+k]*mat_b_T[j*COL_SIZE+k];

```

```

        }
        res[i*COL_SIZE+j] = sum;
    }

}

end_time = chrono::steady_clock::now();
exec_time = end_time-start_time;
ms = exec_time;
cout<<"Execution time:"<<ms.count()<<'\\n';

cout<<"Using 2D Threading (collapsed) \\n";
start_time = chrono::steady_clock::now();
#pragma omp parallel for collapse(2)
for(int i = 0;i<ROW_SIZE;++i){
    for(int j = 0;j<COL_SIZE;++j){
        double sum{0.0};
        for(int k = 0;k<ROW_SIZE;++k){
            sum +=
mat_a[i*COL_SIZE+k]*mat_b_T[j*COL_SIZE+k];
        }
        res[i*COL_SIZE+j] = sum;
    }
}

end_time = chrono::steady_clock::now();
exec_time = end_time-start_time;
ms = exec_time;
cout<<"Execution time:"<<ms.count()<<'\\n';
return 0;
}

```

### Program Output :-

```

● shreyash@shreyash-Aspire-A715-76G:~/UCS645/LAB1$ perf stat ./question2 35.5
Using No Threading
Execution time:487.214
Using 1D Threading
Execution time:106.92
Using 2D Threading
Execution time:105.054
Using 2D Threading (collapsed)
Execution time:104.543

```

## Observation :-

The implementation incorporates matrix transposition to enhance cache utilization and minimize memory latency. We also opted for a single flattened large vector rather than a vector of vectors. Since standard C++ vectors are heap-allocated, a nested vector structure results in non-contiguous memory blocks, which degrades cache performance and increases latency. By flattening the data structure, we ensured contiguous memory access. Furthermore, we observed that naive 2D threading was suboptimal, as OpenMP tends to discourage nested parallel regions, effectively reducing them to 1D threading. To mitigate this, we utilized the OpenMP collapse clause to merge the loop nests, thereby maximizing parallel efficiency.

## Observations from perf stats :-

```
Performance counter stats for './question2 35.5':

    3,511.91 msec task-clock                #    4.172 CPUs utilized
         473      context-switches         #   134.685 /sec
          46      cpu-migrations           #    13.098 /sec
       8,100      page-faults              #    2.306 K/sec
  9,304,870,823    cpu_atom/cycles/         #    2.650 GHz                (22.66%)
 12,353,005,577    cpu_core/cycles/         #    3.517 GHz                (69.56%)
 13,438,607,862    cpu_atom/instructions/   #    1.44 insn per cycle      (26.55%)
 20,731,796,646    cpu_core/instructions/   #    1.68 insn per cycle      (69.56%)
  1,511,826,416    cpu_atom/branches/       #   430.486 M/sec            (26.66%)
  2,337,953,247    cpu_core/branches/       #   665.722 M/sec            (69.56%)
    5,039,616      cpu_atom/branch-misses/  #    0.33% of all branches    (26.62%)
    6,572,066      cpu_core/branch-misses/  #    0.28% of all branches    (69.56%)
    TopdownL1 (cpu_core)                   #   55.9 % tma_backend_bound
                                           #    1.4 % tma_bad_speculation
                                           #    5.7 % tma_frontend_bound
                                           #   37.0 % tma_retiring        (69.56%)
    TopdownL1 (cpu_atom)                   #    2.4 % tma_bad_speculation
                                           #   29.4 % tma_retiring        (26.70%)
                                           #   65.6 % tma_backend_bound
                                           #   65.6 % tma_backend_bound aux
                                           #    2.6 % tma_frontend_bound  (26.67%)

 0.841833621 seconds time elapsed

 3.497793000 seconds user
 0.015885000 seconds sys

shreyash@shreyash-Aspire-A715-76G:~/UCS645/LAB1$
```

**Parallel Utilization:** The metric 4.172 CPUs utilized indicates that the program actively used approximately 4 logical cores on average.

**Improved Computational Intensity:** The IPC (Instructions Per Cycle) is 1.68. This reflects the computational nature of Matrix Multiplication ( $O(N^3)$ ); the CPU performs more arithmetic operations for every piece of data fetched from memory, resulting in better pipeline utilization compared to memory-bound tasks.

**Predictable Control Flow:** The branch miss rate is exceptionally low at 0.28% (approx. 3 million misses out of 1 billion branches). This confirms that the standard triple-loop structure of matrix multiplication is highly predictable for the CPU's branch predictor, minimizing stalls caused by logical decisions.

### Q3. Pi Calculation :-

#### Code :-

```
#include <chrono>
#include <iostream>
#include <ratio>
using namespace std;

int main() {
    int num_steps{static_cast<int>(1e9)};
    double pi{};
    double sum{0.0};
    double step{1.0/num_steps};

    cout<<"With No Threading \n";
    auto start_time{chrono::steady_clock::now()};
    for(int i = 0; i<num_steps; ++i) {
        double x = (i+0.5)*step;
        sum += 4.0/(1.0+x*x);
    }
    pi = step*sum;
    auto end_time{std::chrono::steady_clock::now()};
    auto exec_time{end_time-start_time};
    chrono::duration<double, std::milli> ms{exec_time};
    cout<<"Execution Time:"<<ms.count()<<'\n';
    cout<<"Pi (Serial):"<<pi<<'\n';

    sum = 0.0;
    cout<<"With Threading \n";
    start_time = chrono::steady_clock::now();
    #pragma omp parallel for reduction(+:sum)
    for(int i = 0; i<num_steps; ++i) {
        double x = (i+0.5)*step;
        sum += 4.0/(1.0+x*x);
    }
    pi = step*sum;
    end_time = chrono::steady_clock::now();
    exec_time = end_time-start_time;
    ms = exec_time;
    cout<<"Execution Time:"<<ms.count()<<'\n';
    cout<<"Pi (Parallel):"<<pi<<'\n';
    return 0;}
```

## Program Output :-

```
shreyash@shreyash-Aspire-A715-76G:~/UCS645/LAB1$ perf stat ./question3 35.5
With No Threading
Execution Time:961.898
Pi (Serial):3.14159
With Threading
Execution Time:298.538
Pi (Parallel):3.14159
```

## Observation :-

The parallel implementation of the Pi calculation demonstrated near-linear scalability, achieving an approximate 3.2x speedup by reducing execution time from 961.898ms (serial) to 298.538ms (parallel). This drastic improvement confirms that the workload is compute-bound, allowing the system to fully utilize the available processing cores without the memory bandwidth bottlenecks observed in previous array-based experiments. Additionally, the use of the **#pragma omp parallel for reduction(+:sum)** directive effectively managed data races without the overhead of explicit locks, though a minor discrepancy in the final Pi value was observed due to the non-associative nature of floating-point arithmetic when summing in parallel.

## Observations from perf stat :-

```
Performance counter stats for './question3 35.5':

      3,504.77 msec task-clock                    #    2.772 CPUs utilized
         303      context-switches               #    86.454 /sec
          15      cpu-migrations                 #    4.280 /sec
         164      page-faults                   #   46.793 /sec
    9,207,853,837  cpu_atom/cycles/               #    2.627 GHz          (20.27%)
   12,946,912,896  cpu_core/cycles/              #    3.694 GHz          (72.81%)
   13,444,069,127  cpu_atom/instructions/        #    1.46  insn per cycle (23.74%)
   28,031,682,501  cpu_core/instructions/        #    2.17  insn per cycle (72.81%)
    1,127,766,497  cpu_atom/branches/           #   321.780 M/sec       (23.78%)
    2,344,541,328  cpu_core/branches/           #   668.957 M/sec       (72.81%)
      113,803      cpu_atom/branch-misses/      #    0.01% of all branches (23.83%)
       65,525      cpu_core/branch-misses/      #    0.00% of all branches (72.81%)
    TopdownL1 (cpu_core)                        #   42.2 % tma_backend bound
                                                #    1.2 % tma_bad_speculation
                                                #    8.0 % tma_frontend_bound
                                                #   48.7 % tma_retiring          (72.81%)
    TopdownL1 (cpu_atom)                       #    0.0 % tma_bad_speculation
                                                #   29.4 % tma_retiring          (23.82%)
                                                #   69.4 % tma_backend bound
                                                #   69.4 % tma_backend_bound_aux
                                                #    1.1 % tma_frontend_bound    (23.84%)

    1.264225498 seconds time elapsed

    3.501165000 seconds user
    0.004957000 seconds sys
```

```
shreyash@shreyash-Aspire-A715-76G:~/UCS645/LAB1$
```

**Average Utilization:** The metric 2.772 CPUs utilized reflects the program's hybrid structure. Since the execution includes a significant serial portion (approx. 962ms on 1 core) followed by a brief parallel portion (approx. 298ms on ~8 cores).

**Compute-Bound Efficiency:** The IPC (Instructions Per Cycle) is 2.17. This indicates the workload is Compute-Bound, heavily utilizing the CPU's arithmetic units for floating-point calculations rather than stalling on memory access.

**Predictable Control Flow:** The branch miss rate is negligible at 0.00% (no misses). This confirms that the simple, fixed-iteration loops used for numerical integration are trivial for the CPU's branch predictor to anticipate, ensuring near-perfect pipeline efficiency.